Análisis del motor de ruteo del TMS OLO

Qué existe hoy, qué falta, qué opciones hay y si conviene construir un motor propio · 13 de noviembre de 2026 · Basado en la lectura del repositorio (rama `main`) y en la conversación de esta sesión

1. Respuesta corta

> **¿Tengo un motor de ruteo?** No en el sentido de un optimizador. Lo que hay es un **planificador heurístico**: agrupa pedidos por zona, llena camiones con un empaque voraz y ordena las paradas por vecino más cercano, con distancias en línea recta. Funciona y está en producción en el sandbox, pero no busca la mejor ruta ni minimiza costo.
>
> **¿Puedo crear mi propio motor?** Sí, y es lo que recomiendo, con una condición: el motor es propio en el **modelo** (reglas, costos, restricciones del negocio), pero se apoya en un **solver open source** (OR-Tools o PyVRP) y en un **motor de calles open source** (OSRM, que ya tiene configuración en el repo). Escribir el solver desde cero no conviene, y el propio ADR-005 del proyecto lo rechaza.
>
> **¿Necesito Google?** No es obligatorio. La investigación ya hecha en el repo estima unos 1.700 USD al mes con Google Maps, casi todo por la matriz de distancias. Un OSRM propio cuesta el servidor, estimado ahí mismo en 40–80 USD al mes.

2. Tres cosas distintas que se llaman "routing"

Conviene separarlas, porque el sistema tiene avances distintos en cada una.

Capa	Qué resuelve	Estado en el TMS
1. Motor de calles (camino y matriz)	Distancia y tiempo reales entre dos puntos, y el trazo sobre el mapa.	**Parcial.** El frontend usa OSRM, por defecto el servidor público de demostración. El backend no lo usa: calcula en línea recta.
2. Optimizador de rutas (VRP)	Qué pedido va en qué camión y en qué orden, minimizando costo y respetando restricciones.	**No existe.** Lo sustituyen heurísticas voraces.
3. Planificación y despacho	Ventana de días, viajes, numeración, replan, congelados, entrega al OMS y al WMS.	**Existe y funciona.** Es la parte más madura.

3. Inventario de lo que existe hoy

3.1 Backend de Planificación (lo que corre de verdad)

Vive en `backend/planning/src/`. Un Scheduler ingesta la bandeja cada 10 minutos y replanifica cada hora.

Pieza	Archivo	Cómo lo hace
Qué se planifica	`replan.py`	Pedidos DISP/DISP, por cliente y por día, en la ventana de mañana más 3 días.
Agrupación	`plan_builder.py` (`_clave`)	Mayoreo por zona de entrega. Una tienda (EPA) va en viaje directo.
Asignación a camión	`plan_builder.py` (`_empacar`)	Empaque voraz: peso al 85 %, volumen al 95 %, tope de 16 paradas. Rota la flota propia.

Orden de paradas	`ruta_geo.py` (`ordenar_por_ruta`)	Vecino más cercano desde un depósito fijo de Costa Rica, en línea recta (haversine).
Fecha de salida (T-n)	`ruta_geo.py` (`lead_time_dias`)	1 día más uno extra por cada 70 km en línea recta de la parada más lejana.
Cliente retira y devoluciones	`plan_builder.py`, `recolectas.py`	Cliente retira va sin camión. Las devoluciones se agendan como paradas de recolección.
Viajes congelados	`viajes_liberados.py`	Lo ya liberado a eFlow no se vuelve a armar.
Persistencia	`plan_builder.py`	`route_plans` → `plan_trips` → `plan_stops`, más `orders` y el espejo de eFlow.

3.2 Frontend de Planificación

Vive en `src/pages/planificacion/` y `src/lib/routePlanning/`. Parte es prototipo anterior al backend.

Archivo	Qué hace	Estado
`distance-matrix.ts`	Matriz N×N con OSRM (`/table`). Si falla, usa línea recta × 1,35 y 30 km/h.	En uso
`route-geometry.ts`	Trazo de la ruta por calles con OSRM (`/route`) para el mapa.	En uso
`optimize-stops.ts`	Vecino más cercano. Arranca en el primer pedido de la lista, no en el depósito.	En uso
`plan-automatico.ts`	Un vehículo por zona, flota propia primero. La capacidad está apagada (`USAR_CAPACIDAD = false`).	En uso
`fleet-split.ts`, `capacity-fit.ts`	Reparto voraz del pedido entre varios vehículos por peso y volumen.	En código, apagado por el flag
`time-windows.ts`	Calcula la hora de llegada por parada y marca las que salen de 8:00–19:00. No reordena.	En uso
`lib/routePlanning/google-distance-matrix.ts`	Cliente de la Distance Matrix de Google.	No está conectado a la interfaz
`lib/routePlanning/plan-route.ts`	Vecino más cercano con avisos de ventana y capacidad. El comentario dice que no es un solver real.	Solo lo usa su autoprueba

3.3 Infraestructura y datos que ya sirven para un motor

- **OSRM propio:** hay un `infra/osrm/docker-compose.yml` que descarga el mapa de Costa Rica y se prepara solo. No vi que esté desplegado.
- **Coordenadas:** `orders` trae latitud y longitud de entrega, y los puntos de entrega tienen estado de geocodificación.
- **Flota:** `vehicles` con capacidad de peso y volumen, y la marca de flota propia (`carrier_id IS NULL`).
- **Costos:** tablas de costeo (`tipos_camion`, `costos_fijos`, `costos_variables`, vista de costo variable por km). El planificador no las usa todavía.
- **Zonas con geocerca:** polígonos por zona (`sql/34`) y días de anticipo por zona (`sql/65`).
- **GPS:** Tracking guarda posiciones reales (`tracking_positions`). A futuro sirven para calibrar tiempos de viaje.

4. Brechas concretas

#	Brecha	Consecuencia
1	No hay optimización global	La zona y la capacidad se resuelven por separado y de forma voraz. Nadie minimiza kilómetros ni número de camiones.
2	El backend usa línea recta	El orden de paradas y el T-n ignoran calles, montañas y ríos. En Costa Rica la diferencia entre línea recta y carretera es grande fuera del Valle Central.
3	Lógica duplicada en dos lenguajes	El T-n y el vecino más cercano existen en TypeScript y en Python, con reglas ligeramente distintas (punto de arranque, fuente de distancia, capacidad encendida o apagada).
4	Depósito fijo en el código	`DEPOT_CR` solo sirve para un almacén de Costa Rica. Venezuela y otros almacenes no entran.
5	Solo flota propia	Si no hay flota propia activa, el backend responde 409. No hay segunda opción con transportistas.
6	Sin costo en la decisión	El mandato pide costo por km por tipo de vehículo. Los datos existen, el planificador no los lee.
7	Ventanas de tiempo solo informativas	Se avisa cuando una parada queda fuera de horario, pero la ruta no se reordena para cumplir.
8	Un viaje por camión por día	No se encadenan dos vueltas cortas del mismo camión.
9	OSRM público por defecto	Sin garantía de servicio y con límite de uso. El propio código lo advierte.
10	Sin auditoría de corridas de optimización	El ADR-005 pide guardar entradas y resultados de cada corrida. Hoy solo hay bitácora de replan.

5. Lo que el proyecto ya decidió sobre este tema

- **ADR-005 (estado: Propuesto):** usar una librería madura, recomienda Google OR-Tools con modelo VRPTW, como trabajo asíncrono y con corridas persistidas y reproducibles. Rechaza de forma explícita la heurística casera de vecino más cercano.
- **Decisión 0001 (aceptada):** márgenes de 85 % en peso y 95 % en volumen, y una lista priorizada: jornada del conductor, agrupación geográfica, distancia real, compatibilidad de vehículo y costo.
- **Hoja de ruta de arquitectura, fase 8:** "Routing optimization": evaluar e integrar OR-Tools como trabajo asíncrono.
- **Investigación de mapas (2026-10-20, para decidir):** compara 14 proveedores. La matriz de distancias domina el costo. Propone una opción híbrida con OSRM propio.
- **Mandato del gerente general (2026-09-21):** Planificación genera sola, con prioridad a flota propia y por fecha de entrega, con ruteo dinámico de varias fuentes y costo por km por tipo de vehículo.

> **Lectura:** la dirección ya está escrita y coincide con lo que conversamos. Lo pendiente es ejecutarla y cerrar el ADR-005, que sigue como propuesta.

6. Opciones

6.1 Optimizador (qué pedido va en qué camión y en qué orden)

Opción	Ventajas	Desventajas	Cuándo conviene
A. Google Route Optimization API	Gestionada, poco que construir, tráfico incluido.	Costo por uso, caja negra, reglas propias difíciles de expresar, dependencia del proveedor.	Para salir rápido o como punto de comparación.
B. OR-Tools o PyVRP propio	Sin costo por uso, control total, mismo lenguaje del backend (Python), cubre flota mixta, ventanas, capacidades y costos.	El ajuste y la operación son del equipo. No trae mapa.	**Recomendada.** Coincide con el ADR-005.
C. VROOM	Muy rápido, se integra con OSRM.	Menos flexible con reglas poco comunes. Es un servicio aparte en C++.	Si la velocidad pesa más que las reglas.
D. Heurística propia mejorada	Simple de explicar y mantener.	Peor resultado, y cada regla nueva obliga a rehacerla. Rechazada por el ADR-005.	Solo como respaldo si el solver falla.
E. Producto comercial (Routific, OptimoRoute y similares)	Listo para usar, con interfaz.	Licencia por vehículo, integración con eFlow y OMS igual de costosa, reglas fuera de tu control.	No encaja con un TMS propio.

6.2 Motor de calles y matriz de distancias

Opción	Costo estimado al mes	Notas
OSRM propio	40–80 USD (servidor)	Ya hay configuración en `infra/osrm/`. Sin tráfico en vivo. La operación es del equipo.
Google Maps Platform	≈ 1.700 USD	El más preciso y con tráfico. La matriz sola son ≈ 1.570 USD.
Mapbox	≈ 560 USD	Buena cuota gratuita. Solo la matriz la excede.
Stadia Maps	≈ 80 USD	Basado en Valhalla. Económico y con plan comercial.
Amazon Location Service	Sin calcular	Nativo de AWS, misma facturación y permisos. Falta leer tarifas.

Los montos salen de `docs/tracking/alternativas-mapas.md`, con el supuesto de 1.320 viajes al mes y unas 12 paradas por viaje. Ese supuesto está marcado ahí como pendiente de validar con datos reales.

7. ¿Puedo crear mi propio motor?

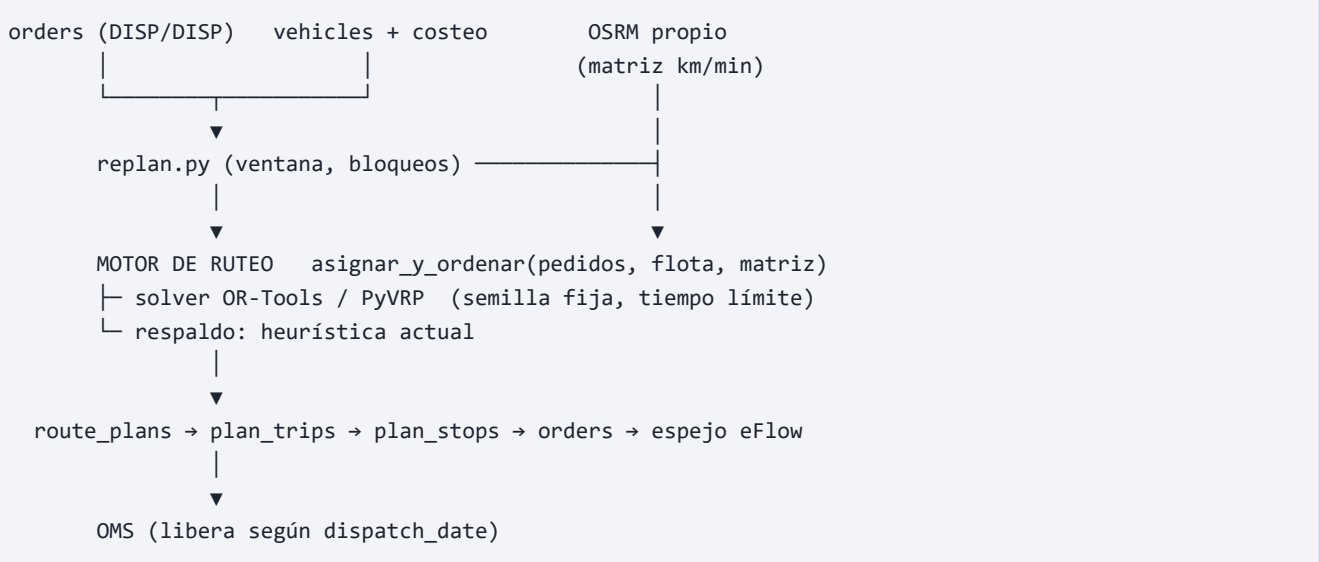
Sí. La pregunta útil es qué parte se construye y qué parte se toma hecha.

Se construye (es lo propio y lo que da valor)	Se toma de código abierto (no reinventar)

• El modelo: qué es un pedido, un vehículo, un depósito y un viaje para OLO. • Las restricciones: margen legal de peso, tope de paradas, tienda en viaje directo, cliente retira, congelados. • La función de costo: fijo + variable + diésel por tipo de camión, y la preferencia por flota propia. • La orquestación: ventana, replan, numeración, entrega al OMS y a eFlow. Ya existe. • La auditoría de cada corrida.	• El solver de optimización (OR-Tools o PyVRP). Son años de investigación. • El grafo de calles y el cálculo de caminos (OSRM con datos de OpenStreetMap). • Los mapas base para dibujar.

> **Por qué es viable aquí:** el volumen es pequeño para un solver. Planificación corre por cliente y por día, con viajes de hasta 16 paradas. Los datos de entrada ya están en la base, y el punto de enganche es una sola función (`_empacar` más `ordenar_por_ruta`).

8. Arquitectura recomendada



Qué se reemplaza y qué se conserva

Pieza actual	Destino	Comentario
`_empacar` (empaque voraz)	Se reemplaza	El solver decide asignación y capacidad a la vez.
`ordenar_por_ruta` (vecino más cercano)	Se reemplaza	El solver devuelve el orden.
Haversine y 70 km por día	Se reemplaza	Pasa a tiempo real de carretera. Afecta el T-n que usa el OMS.
`DEPOT_CR`	Se parametriza	Coordenadas por almacén.
Agrupación por zona o tienda	Se transforma	Pasa a ser una restricción o una agrupación previa.
Márgenes 85 % / 95 % y tope de 16	Se conserva	Como parámetros del modelo.
Consultas de pedidos y flota	Se conserva	Ya entregan lo que el solver necesita.
Cliente retira, congelados, numeración	Se conserva	Fuera del solver o como restricción fija.
Persistencia y replan programado	Se conserva	El contrato con OMS y eFlow no cambia.

Heurísticas del frontend	Se retira con el tiempo	El frontend solo muestra y edita. El cálculo queda en el backend.

9. Plan por fases

1. **Aislar la interfaz.** Envolver la lógica actual en `asignar_y_ordenar(pedidos, flota)`. No cambia el comportamiento.
2. **Desplegar OSRM propio** y usarlo también desde el backend. Deja de depender del servidor público.
3. **Segundo motor con OR-Tools o PyVRP** detrás de un interruptor, con la heurística actual como respaldo.
4. **Comparar en sombra** con días reales de Cofersa: kilómetros, camiones usados, paradas por viaje. Opcional: una corrida contra Google Route Optimization como referencia.
5. **Cambiar el T-n a tiempo real de carretera**, coordinado con el OMS porque mueve la fecha de liberación.
6. **Costos y transportistas terceros**: costo por km por tipo de camión y terceros como segunda opción.
7. **Auditoría de corridas** y cierre formal del ADR-005.

10. Riesgos y decisiones pendientes

Punto	Por qué importa
Estabilidad del replan horario	Un solver con azar puede reordenar viajes que no cambiaron. Hay que fijar semilla y tiempo límite, y no tocar viajes sin cambios.
Calidad de coordenadas	Pedidos sin coordenadas quedan al final de la ruta. El solver no arregla datos malos.
¿Ruteo por cliente o conjunto?	Hoy Cofersa y EPA no comparten camión. Juntarlos ahorra, pero cambia numeración y reglas.
Papel de la zona	Hoy es la unidad de agrupación. Con solver puede ser solo una restricción.
Dónde corre	Lambda alcanza por tamaño de problema. Falta confirmar que la librería entra en el empaquetado actual del despliegue.
Mapa de Venezuela	La configuración de OSRM solo descarga Costa Rica.
Puntos abiertos del mandato	Base del porcentaje de ocupación, margen de utilidad y mecánica de tendering siguen sin definirse y afectan la función de costo.

11. Recomendación

> **Construir un motor propio sobre OR-Tools o PyVRP, con OSRM propio para las distancias, enchufado en el punto donde hoy Planificación empaca y ordena.** Conservar toda la orquestación actual. Usar Google solo como referencia para medir la calidad, no como dependencia permanente.
>
> Primer paso sin riesgo: aislar la interfaz (fase 1) y desplegar OSRM (fase 2). Ninguno cambia el resultado de la planificación actual.

Fuentes revisadas en el repositorio: `backend/planning/src/` (plan_builder, ruta_geo, replan), `src/pages/planificacion/`, `src/lib/routePlanning/`, `infra/osrm/`, `docs/arquitectura-tms-oms/06-adr/ADR-005-routing-engine.md`, `docs/decisions/0001-route-planning-safety-margin-and-optimization.md`, `docs/tracking/alternativas-mapas.md`, `docs/guides/modulo-planificacion.md` y `aidlc/spaces/default/memory/project.md`. No se ejecutó ni modificó código para este análisis. Los costos de proveedores son los de la investigación del 2026-10-20 y pueden haber cambiado.